

Inventory Module

Design & Architecture Reference

C-Edge ERP — Garment Suite | Prepared: 2026-05-04 | Pre-development Document

Inventory Module — Design & Architecture Reference

C-Edge ERP — Garment Suite | Prepared: 2026-05-04 Purpose: Pre-development thinking document. Use this to write Jira epics, user stories, and HTML prototypes. Do NOT start coding until epics are approved.

1. Why Inventory Is Different in a Garment Business

A generic WMS tracks "item → warehouse → qty". A garment ERP cannot. In apparel:

- The same **fabric** can be **free stock**, **reserved against a style**, **assigned to a specific production order**, or **consumed** — all simultaneously from physically different rolls/lots.
- **Lot identity matters** throughout: a lot received today may fail QC three weeks later. Every metre must be traceable back to its GRN, PO, vendor, and lot.
- **Ageing is commercial, not just physical**: fabric sitting unallocated for >90 days locks up working capital. Buyers often own fabric (BOM-reserved) but the factory sits with it — ageing drives follow-up.
- **Three material types behave differently**: Raw Materials (trims, buttons, labels, elastic — non-fabric), Fabric (yardage/meterage, roll-based, has construction + colour + width), and Style Inventory (cut/stitched/finished goods at various WIP stages).
- **Free vs. Allocated stock** is a first-class dimension: planners need to see what is genuinely available to promise vs. what is already spoken for.

2. Core Mental Model — The Inventory Ledger

Everything flows through a single `inventory_ledger` table (append-only, double-entry style). Every stock movement — receipt, reservation, consumption, transfer, return, write-off — is a ledger entry with:

```
(item_id, item_type, lot_id, warehouse_id, movement_type, qty, direction, source_doc_type, source_doc_id,
```

Current stock for any slice is always `SUM(qty WHERE direction=IN) - SUM(qty WHERE direction=OUT)` over that slice's filters. This means:

- There are no `qty` columns to keep in sync across multiple tables.
 - Any view of inventory (free, allocated, per-style, per-lot, per-order) is a query on the same ledger.
 - History is automatically available for ageing, consumption analysis, and audit.
-

3. Item Types

3A. Raw Materials (RM)

Trims, accessories, packing materials, labels, threads, buttons, zips, elastic, lining, interlining.

Key attributes: item code, item name, category, UOM, vendor(s), lead time, reorder level, reorder qty, min stock, max stock.

Source master: `rm_masters` (to be created — currently only `fabric_masters` exists in the tenant schema).

3B. Fabric

Already has `fabric_masters` with construction, width, composition, weight, UOM (metre/yard/kg), multi-UOM support, inventory limits. Fabric also has:

- Roll/Batch identity (Lot)
- Width variation per roll (affects usable yield)
- Shade matching requirement (dye lot consistency)
- Shrinkage % (affects actual vs. theoretical consumption)

3C. Style / Finished Goods Inventory

Semi-finished and finished garments at various WIP stages:

- Cut pieces (stage: CUTTING)
- Stitched / Assembled (stage: SEWING)
- Finished / Packed (stage: FINISHING / PACKING)
- Shipped / Despatched (stage: SHIPPED)

Style inventory is always linked to a Production Order. A "free" style inventory means finished goods with no open delivery note against a Buyer PO.

4. Stock Dimensions — The Allocation Hierarchy

Every unit of stock sits at exactly one level of this hierarchy at any time:

```

LEVEL 0 – Free / Unallocated
  ↓ (Style BOM reservation)
LEVEL 1 – Allocated to Style (style_id set, no production order yet)
  ↓ (Production Order raised from style)
LEVEL 2 – Allocated to Production Order (production_order_id set)
  ↓ (Buyer PO linked to Production Order)
LEVEL 3 – Committed to Buyer PO (buyer_po_id set)
  ↓ (Cutting/Consumption issued)
LEVEL 4 – Consumed / WIP

```

The ledger carries `style_id`, `production_order_id`, and `buyer_po_id` on every entry. A query with all three NULL = free stock. A query with only `style_id` set = style-allocated stock.

5. Lot Tracking

Lots are the physical identity of received goods:

- One GRN line → one lot (system-generated: `LOT-YYYY-NNNN`)
- One lot belongs to exactly one item + one vendor + one GRN
- Lots carry: received date, received qty, accepted qty, warehouse location, expiry (if applicable)
- Lot status: `ACTIVE`, `QC_HOLD`, `QUARANTINE`, `EXHAUSTED`, `WRITTEN_OFF`
- For fabric: lot also carries shade band, roll count, avg width per roll

Lot-level ageing is calculated from `lot.received_date` to `CURRENT_DATE`. Three ageing brackets:

- **Fresh:** 0–60 days
- **Ageing:** 61–120 days
- **Critical:** 121–180 days
- **Dead Stock:** > 180 days

6. Warehouse Structure

```

warehouse (top-level: Factory 1 Store, Factory 2 Store, Central Warehouse)
├── warehouse_zone (RM Zone, Fabric Zone, QC Hold Area, Finished Goods Zone)
│   └── bin_location (Rack A / Shelf 2 / Bin 04 – physical slot for a lot/item)

```

All three levels are required from Phase 1. Bin management is not "future" in a garment factory — fabric rolls and trim boxes are physically placed in specific rack slots and the system must track where each lot lives.

6A. Bin Location Management

A **bin** is the smallest addressable physical slot in a warehouse. In a garment store it might be:

- A rack position for fabric rolls: Rack-A / Bay-3 / Shelf-2
- A shelf bin for trim boxes: Shelf-B / Bin-07
- A pallet position in a finished goods area: FG-Zone / Pallet-14

Why Bin Management Matters

Problem Without Bins	How Bins Solve It
Store keepers waste time searching for a lot	Bin address printed on lot label — find it in seconds
Same item in multiple locations causes pick errors	System shows exactly which bin(s) hold a specific lot
Physical stock counts require walking the entire store	Count sheet sorted by bin address — one zone at a time
FIFO is policy but not enforced physically	Issue Notes specify exact lot + bin — no accidental LIFO picks
Fabric rolls from the same lot get split across locations	Bin shows partial lot qty — consolidation alerts possible

Bin Addressing Scheme

Bin code is a structured string composed of zone + aisle + rack + shelf + bin:

```
{ZONE_CODE}-{AISLE}-{RACK}-{SHELF}-{BIN}
Example:  FAB-A-R03S2-B04   →  Fabric Zone / Aisle A / Rack 03 Shelf 2 / Bin 04
          RM-B-R01S1-B11   →  RM Zone / Aisle B / Rack 01 Shelf 1 / Bin 11
          FG-P14           →  Finished Goods / Pallet 14 (flat structure)
```

The code format is configurable per warehouse — not all warehouses have the same depth of structure.

Bin States

State	Meaning
EMPTY	No stock assigned to this bin
OCCUPIED	Has one or more lot assignments
RESERVED	Pre-assigned for an incoming GRN / transfer (not yet received)
BLOCKED	Temporarily unavailable (pest control, damage, inspection)

Bin Capacity (Optional)

Each bin can carry a `max_capacity_qty` and `max_capacity_uom`. The system warns (not blocks in Phase 1) if an assignment would exceed capacity. Useful for fabric rolls where a shelf physically holds only X rolls.

Lot-to-Bin Assignment

- One lot can span **multiple bins** (e.g. 50 rolls of fabric split across Bin 04 and Bin 05)
- One bin can hold **multiple lots** of the same item (e.g. two trim lots on the same shelf)
- One bin must NOT hold lots of **different items** — the system enforces this (configurable per warehouse)
- The `lot_bin_assignment` table is the join between lots and bins, carrying the qty stored in that specific bin

Bin Movement Types

Two new movement types added to the ledger for bin-level moves that don't change warehouse or zone:

Movement Type	Direction	Meaning
<code>BIN_TRANSFER_OUT</code>	OUT	Lot leaves source bin (within same warehouse zone)
<code>BIN_TRANSFER_IN</code>	IN	Lot arrives at destination bin (within same warehouse zone)

A bin transfer is a **zero net ledger change** at the warehouse level — stock doesn't leave the warehouse. It only updates `lot_bin_assignment` records and posts the two bin-level ledger entries for traceability.

Bin on Issue Notes

When a Cutting Issue Note is created, the system suggests the exact bin(s) to pick from, applying FIFO:

```
oldest lot → bin address for that lot → pick from that bin first
if qty in bin < required → suggest next bin from same lot or next lot
```

The Issue Note line carries: `lot_id`, `bin_location_id`, `pick_qty` — making it a precise pick instruction for the store keeper.

Bin in Stock Count

Physical stock counts are always done **bin by bin**. The count sheet groups rows by bin address. The store keeper walks Bin 01 → Bin 02 → ... and enters the actual qty found in each bin. The system reconciles:

- System qty per bin (from `lot_bin_assignment`) vs. physically counted qty
- Variances are flagged per bin and consolidated per lot before adjustment approval

Bin Label / Barcode Support (Phase 2)

Each bin has a QR / barcode label. Scanning a bin during receiving or counting brings up the bin's current contents. This is Phase 2 — Phase 1 builds the data model and UI; the scan API is added when mobile devices are introduced.

7. Movements — Every Type Defined

Movement Type	Direction	Source Document	Effect
GRN_RECEIPT	IN	grn_header	Stock enters — free or QC hold
QC_PASS	IN	qc_inspection	QC hold → free stock (already modelled in NE-14B)
QC_FAIL_RETURN	OUT	vendor_return	QC fail → leaves store
STYLE_RESERVATION	OUT (virtual)	style_bom	Free → style-allocated (no physical move)
STYLE_UNRESERVATION	IN (virtual)	style_bom	Reversal if style cancelled
PO_ALLOCATION	OUT (virtual)	production_order	Style-allocated → PO-allocated
ISSUE_TO_CUTTING	OUT	cutting_issue_note	Physically leaves store, enters WIP
RETURN_FROM_CUTTING	IN	cutting_return_note	Excess fabric back to store
INTER_WAREHOUSE_OUT	OUT	transfer_note	Leaves source warehouse
INTER_WAREHOUSE_IN	IN	transfer_note	Arrives at destination warehouse
STOCK_ADJUSTMENT_IN	IN	stock_adjustment	Physical count variance (positive)
STOCK_ADJUSTMENT_OUT	OUT	stock_adjustment	Physical count variance (negative)
WRITE_OFF	OUT	write_off_note	Damaged / expired stock removed
VENDOR_RETURN_DISPATCH	OUT	vendor_return	Physical return to vendor
FINISHED_GOODS_IN	IN	production_order	Completed garments enter FG store
FINISHED_GOODS_OUT	OUT	delivery_note	Shipped to buyer
BIN_TRANSFER_OUT	OUT	bin_transfer	Lot leaves source bin (same warehouse, no net change)
BIN_TRANSFER_IN	IN	bin_transfer	Lot arrives at destination bin (same warehouse)

Virtual movements (RESERVATION, UNRESERVATION, ALLOCATION) do not move physical goods but change the "available to promise" calculation.

Bin transfer movements (`BIN_TRANSFER_OUT` + `BIN_TRANSFER_IN`) are always posted as a pair. Net warehouse-level qty = zero. They exist solely for bin-level traceability and to keep `lot_bin_assignment` accurate.

8. BOM (Bill of Materials) — The Bridge Between Style and Inventory

A **Style BOM** defines what RM and Fabric is needed per garment:

```
style_bom_header: style_id, version, status (DRAFT/ACTIVE/SUPERSEDED), created_by, approved_by
style_bom_line:  bom_header_id, item_type (FABRIC/RM), item_id, qty_per_garment, uom_id,
                  wastage_pct, effective_qty_per_garment, colour_id (for fabric), size_group,
                  is_optional, substitute_item_id
```

When a Production Order is raised from a style:

```
required_qty_per_item = style_bom_line.effective_qty_per_garment × production_order.qty
```

This drives:

1. **Material Requirement** — what to procure if not in stock
2. **Reservation** — reserve from free stock against the PO
3. **Issue** — physical issuance to cutting floor

9. Stock Views Required — What the UI Must Show

9A. Raw Material Stock Summary

Filters: warehouse, category, item, date range Columns: Item Code | Item Name | Category | UOM | Free Qty | Style-Allocated | PO-Allocated | Total On-Hand | Reorder Level | Status

9B. Fabric Stock Summary

Filters: warehouse, fabric name/code, colour, construction type, date range Columns: Fabric | Colour | Construction | Width | UOM | Lot Count | Free Qty | Allocated Qty | Total On-Hand | Avg Age (days)

9C. Lot-Wise Stock Ledger

Filters: item, lot number, date range Columns: Lot # | Item | Vendor | GRN # | Received Date | Received Qty | Accepted Qty | Issued Qty | Balance Qty | Age (days) | Ageing Band | Status

9D. Style-Wise Material Allocation

Filters: style, season, buyer Columns: Style Code | Style Name | Buyer | BOM Version | Item | Required Qty | Allocated (Free) | Allocated (PO) | Shortfall | Coverage %

9E. Production Order Material Status

Filters: production order, date range Columns: PO # | Style | Qty | Item | Required | Issued | Balance to Issue | % Issued

9F. Buyer PO–Linked Stock

Filters: buyer, buyer PO number, delivery date range Shows: Which lots/items are committed to a specific Buyer PO and the allocation status.

9G. Free Stock (Available to Promise)

RM and Fabric with no allocation. This is the most operationally critical view.

9H. Ageing Report

Filters: warehouse, item type, ageing band, date Columns: Item | Lot # | Received Date | Age (days) | Ageing Band | Qty | Value (Qty × avg unit cost) | Status Allows writing off or flagging lots from this view.

9I. WIP (Style Inventory) Status

Filters: production order, sewing line, stage Shows garment count at each stage: Cut → Sewing → Finished → Packed → Shipped

9J. Stock Movement Ledger (Audit Trail)

All movements for an item/lot/date range. Full traceability from GRN to consumption.

9K. Bin Inventory View (Where Is Everything?)

Filters: warehouse, zone, aisle, rack, item, lot Columns: Bin Code | Zone | Item | Lot # | Qty in Bin | UOM | Lot Received Date | Age (days) | Bin State Purpose: answers "what is physically sitting in Bin A-R03S2-B04 right now?"

9L. Lot Location Finder

Input: Lot number or Item + Lot Output: list of all bins holding that lot, with qty per bin and bin address Purpose: answers "I have lot LOT-2025-0042 to issue — where do I go to pick it?"

9M. Bin Utilisation Report

Filters: warehouse, zone Columns: Bin Code | State | Items Stored | Total Qty | Max Capacity | Utilisation % | Last Movement Date Purpose: identify empty bins (available for put-away), overcrowded bins, and bins with no movement in >60 days (potential dead stock location alert).

10. Inventory Module — Epic Breakdown

EPIC INV-01: Core Inventory Ledger & Movement Engine

Foundation — must be built first. All other stories depend on this.

What it covers:

- `inventory_ledger` table (append-only, all movement types)
- `warehouse`, `warehouse_zone` tables
- `lot_master` extension (already partially exists — needs full columns)

- Stock calculation service: `getOnHandQty(item, lot, warehouse, asOfDate)`, `getFreeQty()`, `getAllocatedQty()`
- Movement posting API: `POST /inventory/movements` (used internally by GRN, QC, Issue Note services)
- Internal event bus: every movement publishes an event consumed by the stock calculation cache

EPIC INV-02: RM Masters & Minimum Stock Configuration

What it covers:

- `rm_masters` table: item code, name, category, UOM, reorder level, reorder qty, min stock, max stock, lead time, primary vendor
- `rm_categories` master: Trims, Accessories, Packing, Thread, Interlining, etc.
- CRUD API for RM Masters
- Minimum stock alerts (computed: on-hand < reorder level → flag)

EPIC INV-03: Fabric Stock — Lot-Wise Tracking & Roll Management

What it covers:

- Extension of `lot_master` for fabric: shade band, roll count, avg width, shrinkage %
- Fabric-specific stock view: quantity in metres/yards/kg, lot count, shade group
- Fabric inventory receipt (hooks into GRN — NE-13/14A/14B already modelled)
- Shade-matching alert: if two lots from different shade bands are allocated to same style colour

EPIC INV-04: Style BOM Management

What it covers:

- `style_bom_header`, `style_bom_line` tables
- BOM CRUD with version control (DRAFT → ACTIVE → SUPERSEDED)
- BOM explosion: given a style + qty, compute total material requirements
- BOM vs. Stock availability check: show coverage % per item
- BOM approval workflow: DRAFT → ADMIN approve → ACTIVE

EPIC INV-05: Material Reservation & Allocation Engine

What it covers:

- `STYLE_RESERVATION` and `PO_ALLOCATION` virtual movement types
- `POST /inventory/reserve`: reserve free stock against a style or production order
- `DELETE /inventory/reserve`: unreserve (e.g. if PO cancelled)
- Available-to-Promise (ATP) calculation: $\text{free_qty} = \text{on_hand} - \text{reserved} - \text{allocated}$
- Allocation priority rules (FIFO by lot received date — oldest lot allocated first)
- Shortfall detection: if PO qty > available free stock → raise procurement alert

EPIC INV-06: Issue to Production (Cutting Issue Notes)

What it covers:

- `cutting_issue_note_header`, `cutting_issue_note_line` tables
- Issue workflow: store keeper creates issue note against a Production Order → approved by supervisor → physically issued → `ISSUE_TO_CUTTING` ledger entry
- Return from cutting: excess fabric returned → `RETURN_FROM_CUTTING` entry
- Issue vs. BOM comparison: issued qty vs. required qty from BOM (over/under issue alert)
- Consumption efficiency: actual issued vs. theoretical BOM qty

EPIC INV-07: Stock Views & Reporting Dashboard

What it covers:

- All views from Section 9 (9A–9J above)
- Unified inventory dashboard with KPI cards: Total Free RM, Total Free Fabric, Unallocated Value, Ageing > 90 days value
- Export to Excel for all views

EPIC INV-08: Ageing Management & Write-Off

What it covers:

- Ageing calculation: `CURRENT_DATE - lot.received_date` in days
- Ageing bands: Fresh / Ageing / Critical / Dead Stock (configurable thresholds)
- Ageing report with value (qty × avg unit cost from GRN)
- Write-off workflow: USER flags lot → ADMIN approves write-off → `WRITE_OFF` ledger entry
- Automatic ageing alerts: lots crossing into Critical/Dead Stock trigger notification

EPIC INV-09: Inter-Warehouse Transfer

What it covers:

- `transfer_note_header`, `transfer_note_line` tables
- Transfer request: source warehouse clerk raises request → destination warehouse confirms receipt
- `INTER_WAREHOUSE_OUT` + `INTER_WAREHOUSE_IN` ledger pair
- In-transit tracking: between OUT and IN, stock shows as "In Transit" (not in either warehouse)

EPIC INV-10: Physical Stock Count & Adjustment

What it covers:

- `stock_count_header`, `stock_count_line` tables
- Freeze snapshot: take a count of system qty per lot at a point in time
- Enter physical count: actual counted qty per lot
- Variance computation: physical - system = variance
- Adjustment approval: ADMIN approves → `STOCK_ADJUSTMENT_IN/OUT` ledger entry
- Count sheet export to Excel (for offline counting)

EPIC INV-11: Style Inventory / WIP Tracking

What it covers:

- `style_inventory_movement` table: tracks garment units through production stages (CUT, SEW, FINISH, PACK, SHIP)
- Posted from Production Order stage completions
- Style inventory views: how many units at each stage per PO
- Free finished goods: finished units with no Delivery Note raised
- Buyer PO completion: how much of a Buyer PO has been shipped

EPIC INV-12: Bin Location Management

Enables precise physical traceability — where exactly is each lot in the store.

What it covers:

- `bin_location` table: warehouse → zone → bin hierarchy with code, capacity, state
 - `lot_bin_assignment` table: which lot is in which bin, with qty per bin
 - Bin master CRUD: create bins individually or bulk-import from Excel (rack layout sheet)
 - Bin state management: EMPTY → OCCUPIED → BLOCKED transitions
 - Put-away logic: on GRN receipt, system suggests the best available bin (by zone type and capacity) — store keeper confirms or overrides
 - Pick suggestion on Issue Notes: system resolves FIFO lot → bin address → pick qty per bin
 - Bin transfer: move stock within the same warehouse from one bin to another (`BIN_TRANSFER_OUT` + `BIN_TRANSFER_IN` ledger pair)
 - Bin inventory view (9K): current contents of any bin
 - Lot location finder (9L): given a lot, show all bins and qty per bin
 - Bin utilisation report (9M): occupancy, empty bins, stale bins
 - Capacity warning: alert when assignment would exceed `bin.max_capacity_qty`
 - Bin consolidation suggestion: identify bins holding partial qty of the same lot across multiple locations — suggest consolidation to reduce fragmentation
-

11. Database Schema — Key Tables

Core Ledger

```
inventory_ledger (
  id                BIGSERIAL PK,
  item_type         VARCHAR(10) NOT NULL, -- FABRIC | RM
  item_id           BIGINT      NOT NULL, -- FK → fabric_masters or rm_masters
  lot_id            BIGINT,         -- FK → lot_master (null if no lot tracking)
  warehouse_id      BIGINT      NOT NULL, -- FK → warehouse
  warehouse_zone_id BIGINT,         -- FK → warehouse_zone
  movement_type     VARCHAR(30) NOT NULL, -- see movement types above
  direction         VARCHAR(3)  NOT NULL, -- IN | OUT
  qty               DECIMAL(12,3) NOT NULL,
  uom_id            BIGINT      NOT NULL,
  unit_cost         DECIMAL(12,4), -- from GRN line for costing
  style_id          BIGINT,         -- FK → style (if style-allocated)
  production_order_id BIGINT,       -- FK → production_order (if PO-allocated)
  buyer_po_id      BIGINT,         -- FK → buyer_po (if committed)
  source_doc_type   VARCHAR(30),    -- GRN | QC_INSPECTION | ISSUE_NOTE | etc.
  source_doc_id     BIGINT,         -- FK to source document
  source_doc_line_id BIGINT,        -- FK to source document line
  remarks           TEXT,
  created_by        BIGINT,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
  -- NO updated_at, NO soft-delete: ledger is append-only
)
-- Indices: (item_type, item_id), (lot_id), (warehouse_id), (movement_type),
--           (production_order_id), (buyer_po_id), (style_id), (created_at)
```

Warehouse

```
warehouse (
  id                BIGSERIAL PK,
  code              VARCHAR(20) UNIQUE NOT NULL,
  name              VARCHAR(100) NOT NULL,
  factory_id        BIGINT,         -- FK → companies (factory)
  type              VARCHAR(20),    -- RM_STORE | FABRIC_STORE | FG_STORE | CENTRAL
  is_active         BOOLEAN DEFAULT TRUE,
  audit cols...
)

warehouse_zone (
  id                BIGSERIAL PK,
  warehouse_id      BIGINT NOT NULL,
  code              VARCHAR(20),
  name              VARCHAR(100),
  zone_type         VARCHAR(20),    -- RECEIVING | STORAGE | QC_HOLD | DISPATCH
  audit cols...
)
```

RM Master

```
rm_masters (  
  id          BIGSERIAL PK,  
  item_code   VARCHAR(30) UNIQUE NOT NULL,  
  item_name   VARCHAR(150) NOT NULL,  
  category_id BIGINT NOT NULL,    -- FK → rm_categories  
  uom_id      BIGINT NOT NULL,    -- FK → uoms_master  
  description TEXT,  
  reorder_level DECIMAL(12,3),  
  reorder_qty  DECIMAL(12,3),  
  min_stock_qty DECIMAL(12,3),  
  max_stock_qty DECIMAL(12,3),  
  lead_time_days INTEGER,  
  primary_vendor_id BIGINT,      -- FK → vendors_master  
  is_active      BOOLEAN DEFAULT TRUE,  
  audit cols...  
)  
  
rm_categories (  
  id    BIGSERIAL PK,  
  code  VARCHAR(20) UNIQUE NOT NULL,  
  name  VARCHAR(100) NOT NULL  
  -- e.g. TRIMS, ACCESSORIES, PACKING, THREAD, INTERLINING, LABEL, ELASTIC  
)
```

Lot Master (extended)

```
lot_master (  
  id                BIGSERIAL PK,  
  lot_number        VARCHAR(30) UNIQUE NOT NULL, -- LOT-YYYY-NNNN  
  item_type         VARCHAR(10) NOT NULL,        -- FABRIC | RM  
  item_id           BIGINT NOT NULL,  
  vendor_id         BIGINT,  
  grn_id            BIGINT,                      -- FK → grn_header  
  grn_line_id       BIGINT,  
  warehouse_id      BIGINT,  
  warehouse_zone_id BIGINT,  
  received_date     DATE NOT NULL,  
  received_qty      DECIMAL(12,3) NOT NULL,  
  accepted_qty      DECIMAL(12,3),  
  uom_id            BIGINT,  
  unit_cost         DECIMAL(12,4),              -- from GRN line  
  -- Fabric-specific (null for RM)  
  shade_band        VARCHAR(30),  
  roll_count        INTEGER,  
  avg_width_cm      DECIMAL(6,2),  
  shrinkage_pct     DECIMAL(5,2),  
  -- Status  
  status            VARCHAR(20) DEFAULT 'ACTIVE', -- ACTIVE|QC_HOLD|QUARANTINE|EXHAUSTED|WRITTEN_OFF  
  -- Ageing (computed, not stored)  
  expiry_date       DATE,  
  audit cols...  
)
```

Style BOM

```
style_bom_header (  
  id          BIGSERIAL PK,  
  style_id    BIGINT NOT NULL,  
  version     INTEGER NOT NULL DEFAULT 1,  
  status      VARCHAR(20) DEFAULT 'DRAFT',    -- DRAFT | ACTIVE | SUPERSEDED  
  remarks     TEXT,  
  approved_by BIGINT,  
  approved_at TIMESTAMPTZ,  
  audit cols...  
  UNIQUE (style_id, version)  
)  
  
style_bom_line (  
  id          BIGSERIAL PK,  
  bom_header_id BIGINT NOT NULL,  
  line_seq    INTEGER NOT NULL,  
  item_type   VARCHAR(10) NOT NULL,    -- FABRIC | RM  
  item_id     BIGINT NOT NULL,  
  colour_id   BIGINT,                  -- for fabric (colour variant)  
  qty_per_garment DECIMAL(10,4) NOT NULL,  
  uom_id      BIGINT NOT NULL,  
  wastage_pct  DECIMAL(5,2) DEFAULT 0,  
  effective_qty_per_garment DECIMAL(10,4),    -- computed: qty × (1 + wastage/100)  
  size_group   VARCHAR(50),             -- if qty differs by size group  
  is_optional  BOOLEAN DEFAULT FALSE,  
  substitute_item_id BIGINT,            -- FK → rm_masters or fabric_masters  
  notes       TEXT,  
  audit cols...  
)
```

Bin Location & Lot-Bin Assignment

```

bin_location (
  id                BIGSERIAL PK,
  warehouse_id      BIGINT NOT NULL,      -- FK → warehouse
  warehouse_zone_id BIGINT NOT NULL,      -- FK → warehouse_zone
  bin_code          VARCHAR(30) UNIQUE NOT NULL, -- e.g. FAB-A-R03S2-B04
  aisle            VARCHAR(10),
  rack            VARCHAR(10),
  shelf           VARCHAR(10),
  bin_seq          VARCHAR(10),          -- position within shelf
  bin_type         VARCHAR(20),          -- RACK_SHELF | PALLET | FLOOR | CAGE
  max_capacity_qty  DECIMAL(12,3),       -- optional physical limit
  max_capacity_uom_id BIGINT,
  allow_mixed_items BOOLEAN DEFAULT FALSE, -- if true, different items can share this bin
  state            VARCHAR(20) DEFAULT 'EMPTY', -- EMPTY|OCCUPIED|RESERVED|BLOCKED
  blocked_reason   TEXT,                -- reason if state = BLOCKED
  barcode          VARCHAR(50),          -- for future scanner support
  is_active        BOOLEAN DEFAULT TRUE,
  audit cols...
)
-- Index: (warehouse_id), (warehouse_zone_id), (state), (bin_code)

lot_bin_assignment (
  id                BIGSERIAL PK,
  lot_id            BIGINT NOT NULL,      -- FK → lot_master
  bin_location_id   BIGINT NOT NULL,      -- FK → bin_location
  item_type         VARCHAR(10) NOT NULL, -- FABRIC | RM (denormalised for fast queries)
  item_id           BIGINT NOT NULL,
  qty_in_bin        DECIMAL(12,3) NOT NULL, -- current qty stored in this bin for this lot
  uom_id            BIGINT NOT NULL,
  put_away_date     DATE NOT NULL,        -- when stock was placed in this bin
  put_away_by       BIGINT,              -- user who confirmed put-away
  last_pick_date    DATE,                -- last time stock was picked from this bin
  is_active         BOOLEAN DEFAULT TRUE, -- false when qty_in_bin reaches 0
  audit cols...
  UNIQUE (lot_id, bin_location_id)       -- one row per lot-bin pair
)
-- Index: (lot_id), (bin_location_id), (item_type, item_id), (is_active)

```

Key rules for `lot_bin_assignment` :

- `qty_in_bin` is updated on every movement that specifies `bin_location_id`
- When `qty_in_bin` reaches 0: set `is_active = false` ; bin state recalculated (`EMPTY` if all assignments inactive)
- A bin's total occupied qty = `SUM(qty_in_bin WHERE is_active = true AND bin_location_id = ?)`
- `allow_mixed_items = false` (default): INSERT to `lot_bin_assignment` fails if bin already has a different `item_id`

Add `bin_location_id` to the inventory ledger and issue note line:


```
-- inventory_ledger: add column
bin_location_id    BIGINT,    -- FK → bin_location (null for virtual movements)

-- cutting_issue_note_line: add columns
bin_location_id    BIGINT,    -- FK → bin_location (pick from this bin)
pick_qty           DECIMAL(12,3) -- qty to pick from this specific bin (may be < issued_qty if split ad
```

Cutting Issue Note

```
cutting_issue_note_header (
  id                BIGSERIAL PK,
  issue_note_num    VARCHAR(30) UNIQUE NOT NULL, -- CIN-YYYY-NNNN
  production_order_id BIGINT NOT NULL,
  warehouse_id      BIGINT NOT NULL,
  issue_date        DATE NOT NULL,
  status            VARCHAR(20) DEFAULT 'DRAFT', -- DRAFT|PENDING_APPROVAL|APPROVED|ISSUED|CLOSED
  issued_by         BIGINT,
  approved_by       BIGINT,
  remarks           TEXT,
  audit cols...
)

cutting_issue_note_line (
  id                BIGSERIAL PK,
  header_id         BIGINT NOT NULL,
  item_type         VARCHAR(10) NOT NULL,
  item_id           BIGINT NOT NULL,
  lot_id            BIGINT,
  bom_line_id       BIGINT,          -- FK → style_bom_line (for BOM comparison)
  required_qty      DECIMAL(12,3),  -- from BOM explosion
  issued_qty        DECIMAL(12,3),
  uom_id            BIGINT,
  remarks           TEXT
)
```

12. Key Service-Layer Rules

Stock Calculation (ALWAYS query the ledger — never cache counts in item tables)

```
// getFreeQty(itemType, itemId, warehouseId)
SELECT SUM(CASE WHEN direction='IN' THEN qty ELSE -qty END)
FROM inventory_ledger
WHERE item_type = ? AND item_id = ? AND warehouse_id = ?
  AND style_id IS NULL AND production_order_id IS NULL AND buyer_po_id IS NULL
  AND movement_type NOT IN ('STYLE_RESERVATION','PO_ALLOCATION')
```

```
// getAllocatedQty(itemType, itemId, productionOrderId)
SELECT SUM(CASE WHEN direction='IN' THEN qty ELSE -qty END)
FROM inventory_ledger
WHERE item_type = ? AND item_id = ? AND production_order_id = ?
```

FIFO Lot Allocation (for reservations and issues)

When reserving or issuing, always select lots in `received_date ASC` order (oldest first).

```
SELECT * FROM lot_master
WHERE item_type = ? AND item_id = ? AND status = 'ACTIVE'
ORDER BY received_date ASC
```

Ageing (computed at query time — never stored)

```
SELECT lot_number, CURRENT_DATE - received_date AS age_days,
CASE
  WHEN CURRENT_DATE - received_date ≤ 60 THEN 'FRESH'
  WHEN CURRENT_DATE - received_date ≤ 120 THEN 'AGEING'
  WHEN CURRENT_DATE - received_date ≤ 180 THEN 'CRITICAL'
  ELSE 'DEAD_STOCK'
END AS ageing_band
FROM lot_master
```

Dashboard DAO Pattern

The Inventory Dashboard (stock views, ageing report) must use `PlanningDashboardDAO` equivalent — `InventoryReportDAO` annotated `@Repository` with `EntityManager` injection, projecting directly to DTOs. Same CQRS Lite pattern from Backend CLAUDE.md Rule 3.

13. Integration Points (What Already Exists)

Existing Feature	How Inventory Hooks In
GRN Receipt (NE-13, 14A, 14B)	On GRN Approve/QC Pass → POST <code>GRN_RECEIPT</code> / <code>QC_PASS</code> movement to inventory_ledger
Lot Master (already in tenant schema)	Extend columns for fabric-specific fields; use as FK in ledger
Fabric Master (already exists)	item_type='FABRIC', item_id → fabric_masters.id
Vendor Master (already exists)	lot_master.vendor_id, rm_masters.primary_vendor_id
UOM Master (already exists)	All qty columns reference uoms_master
Production Order (NE-21)	production_order_id FK in ledger; PO allocation on order creation
Buyer PO (existing)	buyer_po_id FK in ledger; commitment tracking
Planning Board (NE-24)	On card assignment, check if fabric is reserved for that PO

14. What Does NOT Exist Yet (Must Be Built)

What	Where
<code>inventory_ledger</code> table	New — tenant schema
<code>warehouse</code> , <code>warehouse_zone</code> tables	New — tenant schema
<code>rm_masters</code> , <code>rm_categories</code> tables	New — tenant schema
<code>style_bom_header</code> , <code>style_bom_line</code> tables	New — tenant schema
<code>cutting_issue_note_header/line</code> tables	New — tenant schema
Lot Master extension (shade_band, roll_count etc.)	Extend existing <code>lot_masters</code> via migration
<code>transfer_note_header/line</code>	New — tenant schema
<code>stock_count_header/line</code>	New — tenant schema
<code>write_off_note</code>	New — tenant schema
<code>InventoryMovementService</code>	New Spring service — the central posting engine
<code>InventoryStockService</code>	New — getFreeQty, getAllocatedQty, getOnHandQty methods
<code>InventoryReportDAO</code>	New — CQRS Lite DAO for all report/dashboard queries

15. Development Sequence (Dependency Order)

Build in this order. Each layer unblocks the next.

```
Phase 1 – Foundation (no UI needed yet)
  INV-01  Core Ledger + Movement Engine + Warehouse tables
  INV-02  RM Masters CRUD (simple – unlocks RM stock views)
  Extend  Lot Master (shade band, roll count, unit cost – needs migration)

Phase 2 – Stock Views (UI can start after Phase 1 backend is up)
  INV-03  Fabric Stock Views (lot-wise, ageing)
  INV-07  Stock Reporting Dashboard (calls ledger aggregate queries)
  INV-08  Ageing Report + Write-Off workflow

Phase 3 – BOM & Allocation (depends on PO Master NE-21)
  INV-04  Style BOM Management
  INV-05  Material Reservation & Allocation Engine
  INV-06  Cutting Issue Notes (depends on INV-05)

Phase 4 – Advanced Operations
  INV-09  Inter-Warehouse Transfer
  INV-10  Physical Stock Count & Adjustment
  INV-11  WIP / Style Inventory Tracking
```

16. HTML Prototypes to Build (One per Story Group)

Prototype File	Covers	Key Screens
<code>inventory-dashboard.html</code>	INV-07	KPI cards, stock summary tabs, quick ageing widget
<code>rm-stock.html</code>	INV-02 + INV-07	RM master list, RM stock summary, min stock alerts
<code>fabric-stock.html</code>	INV-03 + INV-07	Fabric stock by lot, lot detail slide-over, shade groups
<code>lot-ledger.html</code>	INV-01 + INV-03	Lot-wise ledger, all movements per lot, ageing band
<code>stock-allocation.html</code>	INV-05	Free vs. allocated view, reserve/unreserve actions
<code>style-bom.html</code>	INV-04	BOM list, BOM detail with lines, BOM vs. stock coverage
<code>cutting-issue.html</code>	INV-06	Issue note creation, BOM-populated lines, issue vs. required
<code>ageing-report.html</code>	INV-08	Ageing table by band, value at risk, write-off modal
<code>warehouse-transfer.html</code>	INV-09	Transfer note, in-transit tracker
<code>stock-count.html</code>	INV-10	Count sheet, system vs. physical, variance approval
<code>wip-tracker.html</code>	INV-11	WIP stage funnel per PO, free FG count
<code>inventory-config.html</code>	INV Config	Tenant inventory settings screen — tier presets + individual flag toggles; ADMIN/SUPER_ADMIN only

17. Open Questions to Resolve Before Development

1. **Does each factory have its own warehouse, or is there a central store?** (Affects warehouse master setup)
2. **Is lot tracking mandatory for all items, or configurable per item/company?** (Already partially answered in NE-14A/B — `company.lotTracking` flag)
3. **Does the system need multi-UOM for RM?** (Fabric already has multi-UOM via `fabric_multiple_uoms` — should RM have the same?)
4. **What is the approval chain for cutting issue notes?** (Store Keeper raises → Supervisor approves → or direct issue for USER role?)
5. **Is style BOM linked to a specific season/collection, or is it per style only?** (Affects BOM versioning)
6. **Should reservation be automatic on Production Order creation (system reserves from free stock), or manual (planner decides what to reserve)?**

7. **Is finished goods (Style Inventory) ever sold without a Buyer PO?** (Determines if free FG inventory needs a dedicated sales order module)
 8. **Should the inventory module track monetary value (FIFO cost, weighted average), or qty only in Phase 1?**
 9. **Inter-company transfers** — if a fabric roll is transferred from Factory 1 to Factory 2, does it stay in the same tenant or cross companies?
 10. **Negative stock** — should the system block issues that would result in negative stock, or warn-only? (Impacts the movement posting service design significantly)
-

18. Configurability — Per-Tenant Feature Flags

The inventory module must serve customers ranging from a small single-warehouse trim store to a large export house with multiple factories, QC labs, and bin-level tracking. The same codebase and the same database schema must support all of them — no forks, no separate builds, no conditionally-compiled editions.

This section defines how configurability is implemented across all three layers: database, backend (Spring Boot), and frontend (React).

18A. The Config Table

`inventory_config` lives in the **tenant schema** (not `public`). One row per tenant, provisioned with safe defaults when a new tenant registers via `TenantDefaultDataService.seedDefaultData()`.

```

inventory_config (
  id                      BIGSERIAL PK,

  -- Lot & Bin
  lot_tracking_enabled    BOOLEAN NOT NULL DEFAULT TRUE,
  bin_management_enabled  BOOLEAN NOT NULL DEFAULT FALSE,

  -- Warehouse
  multi_warehouse_enabled BOOLEAN NOT NULL DEFAULT FALSE,
  default_warehouse_id    BIGINT,                -- FK → warehouse (used when multi_warehouse_enabled)

  -- QC
  qc_mode                 VARCHAR(10) NOT NULL DEFAULT 'INLINE', -- INLINE | QC_HOLD | BOTH

  -- BOM & Reservation
  bom_approval_required   BOOLEAN NOT NULL DEFAULT TRUE,
  reservation_mode        VARCHAR(10) NOT NULL DEFAULT 'MANUAL', -- MANUAL | AUTO
  reservation_on_po_create BOOLEAN NOT NULL DEFAULT FALSE,        -- auto-reserve on PO creation if reservation_mode = AUTO

  -- Issue Notes
  issue_note_approval_required BOOLEAN NOT NULL DEFAULT TRUE,      -- store keeper → supervisor approval required
  allow_negative_stock      BOOLEAN NOT NULL DEFAULT FALSE,        -- block or warn on over-issue

  -- Ageing Thresholds (days) — configurable per customer's commercial norms
  ageing_fresh_days         INTEGER NOT NULL DEFAULT 60,
  ageing_ageing_days        INTEGER NOT NULL DEFAULT 120,
  ageing_critical_days      INTEGER NOT NULL DEFAULT 180,
  -- > ageing_critical_days = DEAD_STOCK

  -- Costing
  costing_method            VARCHAR(15) NOT NULL DEFAULT 'NONE',   -- NONE | FIFO_COST | WEIGHTED_AVG

  -- Bin Rules
  allow_mixed_bin_items     BOOLEAN NOT NULL DEFAULT FALSE,        -- override bin_location.allow_mixed_items

  -- Capacity Enforcement
  bin_capacity_enforce      BOOLEAN NOT NULL DEFAULT FALSE,        -- false=warn, true=hard block on over-capacity

  audit cols...
  -- single-row constraint: only one config row per tenant schema
)

```

Why in the tenant schema, not public? Each tenant's schema is already isolated by `TenantConnectionProvider`. Placing config here means no `tenant_id` filter is ever needed — the schema itself is the boundary. This follows the same rule used for all master tables.

18B. Backend Implementation (Spring Boot 3 / Java 21)

18B-1. Entity & Repository

`InventoryConfig` entity has no `@Table(schema = "public")` — it routes to the active tenant schema automatically.

```
// entity/inventory/InventoryConfig.java
@Entity
@Table(name = "inventory_config")
@Getter @Setter @Builder
public class InventoryConfig {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private boolean lotTrackingEnabled;
    private boolean binManagementEnabled;
    private boolean multiWarehouseEnabled;
    private Long defaultWarehouseId;

    @Enumerated(EnumType.STRING)
    private QcMode qcMode; // INLINE | QC_HOLD | BOTH

    private boolean bomApprovalRequired;

    @Enumerated(EnumType.STRING)
    private ReservationMode reservationMode; // MANUAL | AUTO

    private boolean reservationOnPoCreate;
    private boolean issueNoteApprovalRequired;
    private boolean allowNegativeStock;
    private int ageingFreshDays;
    private int ageingAgeingDays;
    private int ageingCriticalDays;

    @Enumerated(EnumType.STRING)
    private CostingMethod costingMethod; // NONE | FIFO_COST | WEIGHTED_AVG

    private boolean allowMixedBinItems;
    private boolean binCapacityEnforce;
}
```

18B-2. Config Service (Cached per Tenant)

`InventoryConfigService` is a Spring `@Service`. It caches the config for each tenant in a `ConcurrentHashMap<String, InventoryConfig>` (keyed by `tenantSchema`). The cache entry is invalidated when the config is updated via `PUT /api/v1/inventory/config`.


```
// service/inventory/InventoryConfigService.java
@Service
@RequiredArgsConstructor
public class InventoryConfigService {

    private final InventoryConfigRepository configRepository;
    private final ConcurrentHashMap<String, InventoryConfig> cache = new ConcurrentHashMap<>();

    public InventoryConfig getConfig() {
        String schema = TenantContext.getSchema(); // reads ThreadLocal set by JwtAuthentication
        return cache.computeIfAbsent(schema, s →
            configRepository.findFirst()
                .orElseThrow(() → new ResourceNotFoundException("Inventory config not seeded for tenant"
        ));
    }

    public void invalidateCache() {
        cache.remove(TenantContext.getSchema());
    }

    public InventoryConfig update(UpdateInventoryConfigRequest request) {
        InventoryConfig config = getConfig();
        // MapStruct mapper applies request fields to config entity
        inventoryConfigMapper.updateFromRequest(request, config);
        InventoryConfig saved = configRepository.save(config);
        invalidateCache();
        return saved;
    }
}
```

Why `ConcurrentHashMap` and not Spring Cache / Redis? The cache is tenant-scoped and lives in the JVM. `computeIfAbsent` is thread-safe without a lock. A Redis layer would add network latency to every stock movement — the config changes at most a few times a year, so in-memory is the right trade-off for Phase 1.

18B-3. Guard Pattern in Every Service That Has a Flag Dependency

Every service method that depends on a config flag checks it first and throws `ValidationException` with a machine-readable error code if disabled. The controller layer does not check flags — only the service does.

```
// Example: InventoryMovementService — bin assignment on GRN receipt
public void postGrnReceipt(GrnReceiptRequest request) {
    InventoryConfig config = inventoryConfigService.getConfig();

    if (request.getBinLocationId() != null && !config.isBinManagementEnabled()) {
        throw new ValidationException("BIN_MANAGEMENT_DISABLED",
            "Bin assignment is not enabled for this tenant");
    }

    if (request.getLotId() == null && config.isLotTrackingEnabled()) {
        throw new ValidationException("LOT_REQUIRED",
            "Lot tracking is enabled — lotId is required");
    }

    // ... proceed with movement posting
}
```

18B-4. Seeding on Tenant Registration

`TenantDefaultDataService.seedDefaultData()` (already called at registration) gains one new step: seed `inventory_config` with conservative defaults.

```
// In TenantDefaultDataService.seedDefaultData()
private void seedInventoryConfig() {
    InventoryConfig config = InventoryConfig.builder()
        .lotTrackingEnabled(true)
        .binManagementEnabled(false)
        .multiWarehouseEnabled(false)
        .qcMode(QcMode.INLINE)
        .bomApprovalRequired(true)
        .reservationMode(ReservationMode.MANUAL)
        .reservationOnPoCreate(false)
        .issueNoteApprovalRequired(true)
        .allowNegativeStock(false)
        .ageingFreshDays(60)
        .ageingAgeingDays(120)
        .ageingCriticalDays(180)
        .costingMethod(CostingMethod.NONE)
        .allowMixedBinItems(false)
        .binCapacityEnforce(false)
        .build();
    inventoryConfigRepository.save(config);
}
```

This call runs inside a `@Transactional` block with `TenantContext` already set — same pattern as all other seed calls in that method.

18B-5. Config API Endpoints

Guarded by `ADMIN` or `SUPER_ADMIN` role. `SUPER_ADMIN` can update config for any tenant; `ADMIN` can only read/update their own tenant's config.

Method	Path	Auth	Purpose
GET	/api/v1/inventory/config	Any authenticated	Fetch active config for caller's tenant
PUT	/api/v1/inventory/config	ADMIN or SUPER_ADMIN	Update config fields (full replacement)

Response follows the standard `ApiResponse<InventoryConfigDTO>` wrapper.

`InventoryConfigDTO` is the read-safe DTO (never expose the entity directly — Backend CLAUDE.md DTO Pattern). MapStruct mapper: `InventoryConfigMapper`.

18B-6. Flyway Migration

Config table is provisioned in `db/migration/tenant/` (not `public/`) so it is created in every tenant schema when the schema is provisioned.

```
-- db/migration/tenant/V_INV_01__inventory_config.sql
CREATE TABLE IF NOT EXISTS inventory_config (
  id BIGSERIAL PRIMARY KEY,
  lot_tracking_enabled BOOLEAN NOT NULL DEFAULT TRUE,
  bin_management_enabled BOOLEAN NOT NULL DEFAULT FALSE,
  multi_warehouse_enabled BOOLEAN NOT NULL DEFAULT FALSE,
  default_warehouse_id BIGINT,
  qc_mode VARCHAR(10) NOT NULL DEFAULT 'INLINE',
  bom_approval_required BOOLEAN NOT NULL DEFAULT TRUE,
  reservation_mode VARCHAR(10) NOT NULL DEFAULT 'MANUAL',
  reservation_on_po_create BOOLEAN NOT NULL DEFAULT FALSE,
  issue_note_approval_required BOOLEAN NOT NULL DEFAULT TRUE,
  allow_negative_stock BOOLEAN NOT NULL DEFAULT FALSE,
  ageing_fresh_days INTEGER NOT NULL DEFAULT 60,
  ageing_ageing_days INTEGER NOT NULL DEFAULT 120,
  ageing_critical_days INTEGER NOT NULL DEFAULT 180,
  costing_method VARCHAR(15) NOT NULL DEFAULT 'NONE',
  allow_mixed_bin_items BOOLEAN NOT NULL DEFAULT FALSE,
  bin_capacity_enforce BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_by BIGINT,
  updated_by BIGINT
);
```

The migration creates the table; `seedInventoryConfig()` inserts the single row. This separation keeps migrations idempotent (the table creation is safe on re-run) while the data seeding runs only once at tenant registration time.

18C. Frontend Implementation (React 19 / TypeScript)

18C-1. Config Type

```
// src/types/inventory.ts
export interface InventoryConfig {
  lotTrackingEnabled: boolean;
  binManagementEnabled: boolean;
  multiWarehouseEnabled: boolean;
  defaultWarehouseId: number | null;
  qcMode: 'INLINE' | 'QC_HOLD' | 'BOTH';
  bomApprovalRequired: boolean;
  reservationMode: 'MANUAL' | 'AUTO';
  reservationOnPoCreate: boolean;
  issueNoteApprovalRequired: boolean;
  allowNegativeStock: boolean;
  ageingFreshDays: number;
  ageingAgeingDays: number;
  ageingCriticalDays: number;
  costingMethod: 'NONE' | 'FIFO_COST' | 'WEIGHTED_AVG';
  allowMixedBinItems: boolean;
  binCapacityEnforce: boolean;
}
```

18C-2. Config Hook (TanStack React Query)

```
// src/hooks/useInventoryConfig.ts
import { useQuery } from '@tanstack/react-query';
import { inventoryApi } from '@api/inventory';

export function useInventoryConfig() {
  return useQuery({
    queryKey: ['inventory-config'],
    queryFn: inventoryApi.getConfig,
    staleTime: 10 * 60 * 1000, // 10 min – config changes rarely
    gcTime: 30 * 60 * 1000,
  });
}
```

The `queryKey` is tenant-scoped implicitly — the Axios interceptor attaches the JWT (which carries `tenantSchema`) on every request, so each tenant's JWT session returns its own config from the backend.

18C-3. Conditional UI Pattern

Config is consumed at the component level using the hook. Fields, columns, and nav items that depend on a flag are conditionally rendered — never just hidden with CSS (hidden DOM still submits form values).

```
// Example: GRN Receipt line – LOT and BIN fields
const { data: config } = useInventoryConfig();

{config?.lotTrackingEnabled && (
  <MasterFormField
    label="Lot Number"
    fieldType="text"
    required
    {...register('lotNumber')}
    error={errors.lotNumber?.message}
  />
)}

{config?.binManagementEnabled && (
  <MasterFormField
    label="Put-Away Bin"
    fieldType="dropdown"
    options={binOptions}
    required
    control={control}
    name="binLocationId"
    error={errors.binLocationId?.message}
  />
)}
```

Columns in stock views (lot #, bin code, costing value) are likewise conditionally included in the table column definition array before rendering — not toggled with `hidden` class.

18C-4. Config Admin Screen

A dedicated settings screen at `/settings/inventory-config` (guarded to `ADMIN` and `SUPER_ADMIN`) allows updating all flags via a form. Uses `React Hook Form + Zod` resolver, `MasterFormField` for all fields. Submit calls `PUT /api/v1/inventory/config` . On success, React Query's `invalidateQueries(['inventory-config'])` flushes the frontend cache so the updated config is fetched on next use.

Prototype file: `inventory-config.html` (to be built — see Section 16 update below).

18D. Progressive Tier Model (Onboarding Reference)

These are the three common customer profiles. The `SUPER_ADMIN` sets flags during tenant onboarding via the config screen. Defaults already match Tier 1 — so a new tenant gets a working system without any configuration.

Flag	Tier 1 — Simple Store	Tier 2 — Mid-Size Factory	Tier 3 — Large Export House
<code>lot_tracking_enabled</code>	<code>false</code>	<code>true</code>	<code>true</code>
<code>bin_management_enabled</code>	<code>false</code>	<code>false</code>	<code>true</code>
<code>multi_warehouse_enabled</code>	<code>false</code>	<code>false</code>	<code>true</code>
<code>qc_mode</code>	<code>INLINE</code>	<code>INLINE</code>	<code>BOTH</code>
<code>bom_approval_required</code>	<code>false</code>	<code>true</code>	<code>true</code>
<code>reservation_mode</code>	<code>MANUAL</code>	<code>MANUAL</code>	<code>AUTO</code>
<code>issue_note_approval_required</code>	<code>false</code>	<code>true</code>	<code>true</code>
<code>allow_negative_stock</code>	<code>true</code>	<code>false</code>	<code>false</code>
<code>costing_method</code>	<code>NONE</code>	<code>NONE</code>	<code>WEIGHTED_AVG</code>
<code>bin_capacity_enforce</code>	<code>false</code>	<code>false</code>	<code>true</code>

Tier 1: GRN → stock in → issue out. No lots, no bins, no approvals. Staff just record what came in and went out.

Tier 2: Full lot traceability, ageing reports, FIFO allocation, BOM-driven issue notes with supervisor approval.

Tier 3: All of the above plus multi-warehouse with inter-warehouse transfers, bin-level put-away and pick instructions, auto-reservation on PO creation, weighted average costing, bin capacity hard enforcement.

18E. What Changes Per Flag — Full Impact Matrix

Flag	Backend Impact	Frontend Impact
<code>lot_tracking_enabled = false</code>	<code>lot_id</code> not required on movement posts; <code>lot_master</code> not created on GRN receipt	Lot # column hidden in all stock views; LOT field removed from GRN and issue note lines
<code>lot_tracking_enabled = true</code>	<code>lot_id</code> required on all physical movements (GRN_RECEIPT, ISSUE_TO_CUTTING, QC_PASS); <code>lot_master</code> row auto-created on GRN receipt	Lot # column visible; LOT field required in forms; ageing report available
<code>bin_management_enabled = false</code>	<code>bin_location_id</code> ignored on all movement posts; put-away step skipped after GRN approval	Bin column hidden; bin assignment step removed from GRN flow; views 9K/9L/9M not shown in nav
<code>bin_management_enabled = true</code>	<code>bin_location_id</code> required on physical movements; <code>lot_bin_assignment</code> updated on every movement; bin state recalculated	Bin code column shown; put-away UI shown post-GRN; pick suggestion shown on issue notes; bin views 9K/9L/9M in nav
<code>multi_warehouse_enabled = false</code>	All movements default to <code>default_warehouse_id</code> ; warehouse param optional on API calls; inter-warehouse transfer endpoints return <code>403</code>	Warehouse selector hidden on all documents; transfer nav item not shown
<code>multi_warehouse_enabled = true</code>	Warehouse required on all movement posts; inter-warehouse transfer endpoints active	Warehouse selector shown; transfer module in nav
<code>qc_mode = INLINE</code>	Only <code>acceptedQty</code> / <code>rejectedQty</code> inline fields on GRN lines; <code>QC_HOLD</code> movement type blocked	GRN shows inline accept/reject fields; QC Inspection page not in nav

Flag	Backend Impact	Frontend Impact
<code>qc_mode = QC_HOLD</code>	GRN posts full qty as <code>QC_HOLD</code> ; inline accept/reject fields blocked	GRN has no inline QC fields; QC Inspection page shown in nav
<code>qc_mode = BOTH</code>	GRN line has a QC mode toggle: INLINE or HOLD per line	Toggle shown on each GRN line; both inspection page and inline fields available
<code>bom_approval_required = false</code>	BOM status skips DRAFT → goes directly to ACTIVE on save	No approval button/status shown in BOM form
<code>reservation_mode = AUTO</code>	On PO creation, system immediately calls reservation engine for all BOM lines	"Reserve" button hidden; stock coverage shown as auto-calculated
<code>reservation_mode = MANUAL</code>	No auto-reservation; planner explicitly calls <code>POST /inventory/reserve</code>	"Reserve" button visible on production order detail
<code>issue_note_approval_required = false</code>	Issue note goes from DRAFT directly to ISSUED on submit	No "Submit for Approval" step; direct "Issue" button
<code>allow_negative_stock = false</code>	<code>InventoryMovementService.postMovement()</code> throws <code>ValidationException("INSUFFICIENT_STOCK")</code> if free qty < issued qty	Issue Note shows real-time stock warning; Issue button disabled when qty insufficient
<code>allow_negative_stock = true</code>	Movement posts regardless of current balance; warning logged	Warning badge shown on issue note but Issue button remains enabled
<code>costing_method = NONE</code>	<code>unit_cost</code> not stored on ledger; no value columns computed	Value column hidden in ageing report and stock views

Flag	Backend Impact	Frontend Impact
<code>costing_method = WEIGHTED_AVG</code>	Weighted avg unit cost recalculated on every GRN receipt and write-off	Value column visible; ageing report shows stock value at risk
<code>bin_capacity_enforce = false</code>	Over-capacity bin assignment logs a warning but posts	Capacity warning badge shown on bin selector
<code>bin_capacity_enforce = true</code>	Over-capacity bin assignment throws <code>ValidationException("BIN_CAPACITY_EXCEEDED")</code>	Bin selector disables over-capacity bins; hard error shown

18F. Multi-Tenancy Alignment — Rules to Follow

These rules align with the existing multi-tenancy architecture (Backend CLAUDE.md — Multi-Tenancy Strategy):

- `inventory_config` has NO `tenant_id` column. The schema is the isolation boundary. `TenantConnectionProvider` routes all queries to the correct tenant schema. Adding a `tenant_id` filter would be an anti-pattern.
- `InventoryConfigService` reads `TenantContext.getSchema()` for cache keying — not `TenantContext.getTenantId()`. This is the only place schema is read directly; all other inventory services go through `inventoryConfigService.getConfig()`.
- Seeding runs inside `TenantDefaultDataService.seedDefaultData()` with `TenantContext` pre-set. Never seed from outside this method — it is the single entry point for all per-tenant defaults.
- Config migration lives in `db/migration/tenant/`, not `db/migration/public/`. This ensures it is applied to every tenant schema via `TenantSchemaService.provisionTenantSchema()`.
- `SUPER_ADMIN` can read and update any tenant's config by using their cross-tenant access. `ADMIN` can only operate on their own schema (enforced naturally — their JWT routes all queries to their schema).
- The config endpoint `GET /api/v1/inventory/config` is NOT under `/masters/` — it is a module-level setting, not a master data record. Path: `/api/v1/inventory/config`.
- Cache invalidation is tenant-scoped. When `PUT /api/v1/inventory/config` is called, only the calling tenant's entry is evicted from `InventoryConfigService.cache`. Other tenants' cached configs are unaffected.
- All timestamps follow the UTC storage rule: `created_at`, `updated_at` stored as `TIMESTAMPZ` in PostgreSQL UTC. Serialised to IST at the Jackson layer — no changes needed in `inventory_config` beyond the standard audit column pattern.

18G. SUPER_ADMIN Onboarding Screen

When SUPER_ADMIN provisions a new tenant, a dedicated onboarding wizard step (after tenant registration) sets the inventory tier. Options:

- **Quick Setup** — SUPER_ADMIN selects a tier (Simple / Mid-Size / Large) and the backend applies the matching preset values from Section 18D.
- **Custom Setup** — SUPER_ADMIN sets each flag individually.

Backend: `POST /api/v1/inventory/config/apply-preset` body `{ "preset": "TIER_1" | "TIER_2" | "TIER_3"`
`}` — reads the preset map and applies it via the existing `update()` method. This endpoint is `SUPER_ADMIN` only.

Section 18 added 2026-05-04. Review with backend team before writing INV-01 Jira stories — the config table must be provisioned before any other inventory table is built.